interviewers with opposite positions, one considering parallelization “cheating” to solve the problem we were discussing, and another one expecting the interviewee to suggest parallelization to solve it. Keep this in mind in your next interview - of course, it also depends on the specific problems and on where the interviewer wants to lead you, but **asking the interviewer** about your options for multi-threading and parallelization is often a good idea (provided you know what you are talking about!)

To give you an idea of the kind of speed-up we are talking about, I witnessed an application’s running time go down from 2 hours to less than 5 minutes by leveraging *Kubernetes* and *Airflow* to distribute data download and processing into small chunks, instead of processing the same data sequentially. Of course, splitting data and processing each chunk separately is not always possible, it depends on the domain and on the algorithm.

So, this is the point where we ask ourselves: is clustering such a domain where we can parallelize execution and get away with it?

Can we break down our datasets and apply the algorithms we have discussed in chapter 12 to each partition independently?

13.1.1 Parallel vs Distributed

Before we get to the grain, a disclaimer is due: usually with the term *parallel computing* we only address computations that run on multiple CPUs on the same system – multi-